OBJECT ORIENTED ANALYSIS AND DESIGN

ASSIGNMENT — CAMPUS HOSTEL BOOKING SYSTEM

Group Members

#	Name	Reg No.
1	Armour Peter	IN13/00224/24
2	Mordecai Ominde	IN13/00122/24
3	Brian Ogero	IN13/00139/24
4	Brendah Shahntel	IN13/00172/24
5	Diana Bisase	IN13/00040/24
6	Job Neko Nyayiemi	IN13/00129/24

Task 1: Requirements

Write 5 testable requirements each starting with "The system shall". State allocation priority rules and other assumptions needed to make rules clear.

Requirements

1. The system shall allow a student to only apply for room opening in one semester within the stipulated opening and closing application dates. Applications outside application opening and closing and multiple applications rejected.
2. The system shall return the room to the pool if the student application confirmed deadline reaches and put to Lapsed state when the student fails to confirm application.
3. The system shall prevent the allocation of rooms in state under repair to avoid their booking by eligible students making sure rooms under repair are not booked.
4. **Functional Requirement 2: Automated Allocation Priority Calculation**
   The system shall automatically calculate a hostel allocation priority score for every student that is eligible based on the five factors: year of study, hostel accommodation distance from home, previous hostel status, fee payment status & disciplinary records. Each factor must contribute to number of points awarded under each category. If it's first time student receive additional priority points for fair distribution for accommodation, and for the travelling students, higher priority is offered due to challenges associated with travelling.
5. The system shall store the calculated scores & use them during allocation process.

Testing Procedures

1. The system should calculate an allocation score using all five factors.
2. The calculated score should not exceed the maximum possible score of 100 points.
3. Applicants should receive scores reflecting the agreed scoring rules based on their characteristics.
4. The system should correctly rank applicants according to their total score.

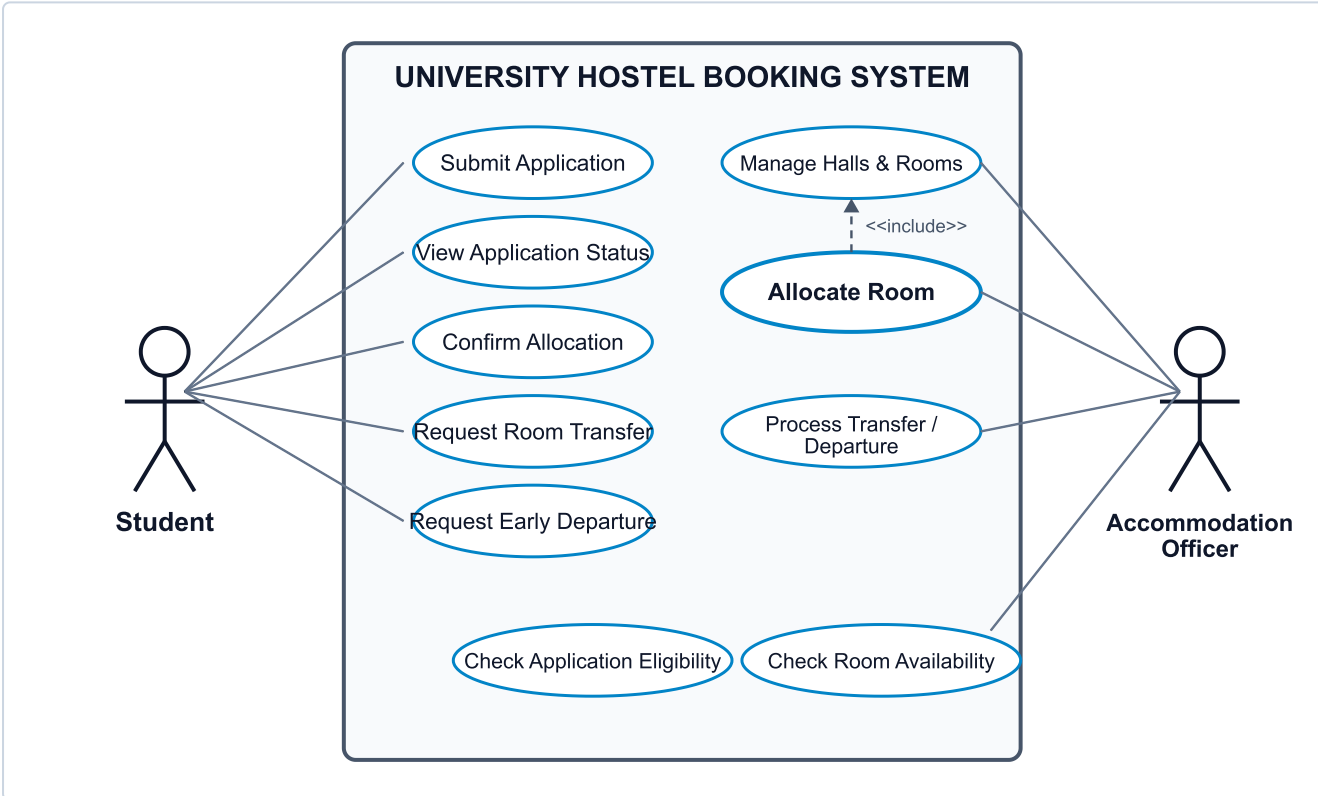
Conclusion

The requirements, allocation priority rules and adopted assumptions provide a clear foundation for the campus Hostel Booking System. They establish how students apply, how accommodation priority is determined, and how available rooms are allocated fairly. These rules will guide the development of the UML diagrams and ensure consistency throughout the system design.

Task 2: Use Case Diagram & Specification

Draw the diagram with actors and a system boundary. Write one use case with its actors, preconditions, main steps, and two exception flows.

Use Case Diagram



Use Case Description: Allocate Room

Use Case Name: Allocate Room

Primary Actor: Accommodation Officer

Supporting Actor: Student

a) Preconditions

- The hostel application period has closed.
- Students have submitted their applications.
- Hall and room records have been updated with their capacities, room types, and gender restrictions.
- Allocation priority rules have been established.
- Available rooms have been identified excluding rooms under repair.

b) Main Success Steps

1. The accommodation Officer logs into the system.
2. Officer initiates the room allocation process.
3. System retrieves all eligible student applications.
4. System retrieves available rooms from the registered university halls.
5. System ranks applications according to established allocation rules.
6. The system matches applications to suitable rooms.
7. The system generates/allocates suitable rooms to eligible applicants.
8. The system then produces allocation offers for applicants.
9. The system sends notification to student of their allocation offers.

c) Exception Flows

1. **No suitable rooms available:** The system determines that there are no suitable available rooms for an applicant.
2. **Rooms become unavailable during allocation:** The system identifies that a room selected for allocation has been marked as under repair or unavailable.

Task 3: Class Diagram & Design Decision

Show the main classes, key attributes, operations, and association multiplicities. Explain whether Allocation should be a class or an attribute of Student.

Student

```
- String studentId
- String name
- String email
- Gender gender
- int academicYear
```

```
+ submitApplication()
+ viewApplicationStatus()
+ confirmApplication()
```

Application

```
- String applicationId
- String semester
- DateTime submittedAt
- String preferredType
- RoomType roomType
- ApplicationStatus status
```

```
+ submit()
+ checkEligibility()
+ updateStatus()
```

Allocation

```
- String allocationId
- Date allocationDate
- DateTime confirmedAt
- AllocationStatus status
```

```
+ confirm()
+ cancel()
+ transfer()
+ complete()
```

Room

```
- String roomId
- String roomNo
- RoomType roomType
- int capacity
- RoomState state
```

```
+ isAvailable()
+ hasCapacity()
```

Hall

```
- String hallId
- String name
- GenderPolicy policy
```

```
+ addRoom()
+ updateDetails()
+ getRooms()
```

Waiting List Entry

```
- String entryId
- DateTime dateAdded
- int priorityPosition
- WaitingStatus status
```

```
+ calculatePosition()
+ updatePosition()
+ removeFromList()
```

Design Decision: Allocation Class vs. Attribute

- Allocation should be made as a separate class rather than an attribute of Student.
- This is because an allocation has its own properties like allocation ID, confirmation date. It also has its own operations including confirming, cancelling, transferring, and completing.
- Therefore a separate Allocation class provides better organization and flexibility.

Task 4: Behaviour Diagrams

a) Sequence Diagram — Allocating Rooms when demand exceeds capacity

```
Participants: Student, Officer, Application, Room, Allocation, WaitingList 1. Student -> Officer: Submit Application 2. Officer -> Application: Retrieve Applications 3. Application -> Room: Check Availability 4. Room --> Application: Room Available / Not Available 5. Application -> Allocation: Create Allocation (Officer) 6. Allocation -> Student: Confirm Allocation 7. Allocation -> Allocation: Update Status alt [Room Available] Allocation -> Student: Notify Offer alt [No Room Available] Application -> WaitingList: Add student to waiting list WaitingList -> Student: Notify Student
```

b) State Diagram — Allocation Lifecycle

```
[Offered] ---> (Student Confirms) ---> [Confirmed] ---> [Occupied] ---> (Semester Ends) ---> [Completed] | | +---> (Deadline 48 hrs / Student Declines) ---> [Lapsed / Cancelled] ---> [Released]
```

Task 5: Consistency Check

The sequence diagram, class diagram, and state diagram are consistent because they describe the same room allocation process in different ways. The sequence diagram shows how objects communicate, with the Application class checking eligibility and the Room class checking availability, capacity, and repair status. The Allocation class manages room confirmation, cancellation, and completion, while the Waiting List Entry class manages unallocated students. The state diagram shows allocation changes from offered to confirmed, lapsed, or released. Students must confirm their offers within 48 hours; otherwise, the allocation lapses and the room is released for another eligible student. Together, the diagrams ensure proper room allocation and management.